

Complete Study Guide: RNN, LSTM, and GRU

Machine Learning - 3rd Year Final Exam Preparation

Table of Contents

1. Recurrent Neural Networks (RNN) - Complete Theory
 2. LSTM (Long Short-Term Memory) - Complete Theory
 3. GRU (Gated Recurrent Unit) - Complete Theory
 4. Step-by-Step Examples with Dataset
 5. Answers to Exam Questions
 6. Mathematical Problems for Practice
-

1. Recurrent Neural Networks (RNN)

1.1 What is RNN?

RNN is a type of neural network specifically designed to work with **sequential data** where the order and context matter. Unlike feedforward neural networks (like ANNs and CNNs), RNNs have memory that allows them to remember previous inputs.

Key Characteristics:

- **Sequential Data Processing:** Works with time-series data, text, speech, video
- **Memory Feature:** Has hidden state that carries information through time
- **Recurrent Connections:** Output from previous time step feeds back as input
- **Variable Length Input:** Can handle sequences of any length

Difference from ANN and CNN:

- **ANN:** Works with tabular data (independent samples)
- **CNN:** Works with images (spatial patterns)
- **RNN:** Works with sequential data (temporal patterns)

1.2 RNN Architecture

Basic Structure:

Input Layer → Hidden Layer (with recurrence) → Output Layer

At each time step t :

- Input: x_t (current input)
- Hidden State: h_t (memory from previous steps)
- Output: o_t (prediction at current step)

Mathematical Formulation:

Hidden State Update:

$$h_t = f(W_h \cdot x_t + U_h \cdot h_{t-1} + b_h)$$

Output:

$$o_t = g(W_o \cdot h_t + b_o)$$

Where:

- W_h : Weight matrix for input-to-hidden
- U_h : Weight matrix for hidden-to-hidden (recurrent weights)
- W_o : Weight matrix for hidden-to-output
- b_h, b_o : Bias terms
- f : Activation function (typically tanh or ReLU)
- g : Output activation (sigmoid for binary, softmax for multi-class)

1.3 Types of RNN Architectures

1. **Many-to-One**: Sequence input → Single output
 - Example: Sentiment analysis (text → sentiment score)
2. **One-to-Many**: Single input → Sequence output
 - Example: Image captioning (image → sequence of words)
3. **Many-to-Many (same length)**: Sequence input → Sequence output (same length)
 - Example: Parts of speech tagging
4. **Many-to-Many (different length)**: Sequence input → Sequence output (different lengths)
 - Example: Machine translation (English → Bengali)

1.4 Forward Propagation in RNN

Step-by-Step Process:

1. **Initialize:** $h_0 = 0$ (or random small values)
2. **For each time step $t = 1, 2, \dots, T$:**
 - Compute hidden state:

$$h_t = \tanh(W_h \cdot x_t + U_h \cdot h_{t-1} + b_h)$$

- Compute output:

$$o_t = \text{softmax}(W_o \cdot h_t + b_o)$$

3. **Calculate Loss** (for all time steps):

$$L = \sum_t L(o_t, y_t)$$

Where L is typically Cross-Entropy loss

1.5 Backpropagation Through Time (BPTT)

Concept: Since RNN unfolds through time, we need to backpropagate errors through all time steps.

Algorithm:

1. **Forward pass:** Compute all h_t and o_t for $t = 1$ to T
2. **Compute gradients** at output layer (time $T \rightarrow$ time 1):

$$\delta_t = \partial L / \partial o_t$$

3. **Backpropagate through time:**

$$\begin{aligned}\partial L / \partial W_h &= \sum_t (\partial L / \partial h_t) \cdot x_t^T \\ \partial L / \partial U_h &= \sum_t (\partial L / \partial h_t) \cdot h_{t-1}^T \\ \partial L / \partial W_o &= \sum_t (\partial L / \partial o_t) \cdot h_t^T\end{aligned}$$

4. **Update weights** using gradient descent:

$$\begin{aligned} W_h &:= W_h - \eta \cdot \partial L / \partial W_h \\ U_h &:= U_h - \eta \cdot \partial L / \partial U_h \\ W_o &:= W_o - \eta \cdot \partial L / \partial W_o \end{aligned}$$

1.6 Problems with Standard RNN

Problem 1: Vanishing Gradient Problem

What happens:

- During BPTT, gradients are multiplied many times
- If gradient < 1 , it becomes exponentially smaller over time
- Network cannot learn long-term dependencies

Mathematical Explanation:

$$\partial L / \partial h_1 = \partial L / \partial h_T \cdot \partial h_T / \partial h_{\{T-1\}} \cdot \dots \cdot \partial h_2 / \partial h_1$$

If each derivative ≈ 0.1 , after 100 timesteps: $(0.1)^{100} \approx 0$

Consequence: RNN forgets information from early timesteps

Problem 2: Exploding Gradient Problem

What happens:

- If gradients > 1 , they grow exponentially
- Weights become very large
- Training becomes unstable

Solution Approaches:

1. Gradient clipping (limit max gradient value)
2. Better weight initialization
3. Skip connections
4. **LSTM/GRU architectures** (most effective)

1.7 Applications of RNN

1. Natural Language Processing (NLP):

- Language modeling

- Machine translation
- Text generation
- Named entity recognition

2. Time Series Analysis:

- Stock price prediction
- Weather forecasting
- Sensor data analysis

3. Speech Recognition:

- Voice-to-text conversion
- Voice command systems

4. Video Analysis:

- Action recognition
- Video captioning

5. Music Generation:

- Melody composition
 - Music style transfer
-

2. LSTM (Long Short-Term Memory)

2.1 Why LSTM?

Motivation: Standard RNNs suffer from vanishing/exploding gradients and cannot capture long-term dependencies. LSTM was designed to solve these problems.

Key Innovation:

- Instead of a simple hidden state, LSTM has a **cell state** (C_t) that acts as a memory highway
- **Gates** control information flow (what to forget, what to remember, what to output)

2.2 LSTM Architecture

Components:

1. **Cell State (C_t):** Long-term memory (runs through entire chain)
2. **Hidden State (h_t):** Short-term memory (output at each step)
3. **Three Gates:**

- **Forget Gate (f_t):** Decides what to remove from cell state
- **Input Gate (i_t):** Decides what new information to add
- **Output Gate (o_t):** Decides what to output

2.3 LSTM Mathematical Formulation

At each time step t , given input x_t and previous hidden state h_{t-1} :

1. Forget Gate (decides what to forget from C_{t-1}):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Output: Values between 0 and 1
- 0 = completely forget, 1 = completely remember

2. Input Gate (decides what new information to add):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Candidate Values (new information to potentially add):

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

3. Update Cell State:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

- $\odot$ represents element-wise multiplication
- First part: forget old information
- Second part: add new information

4. Output Gate (decides what to output):

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

5. Hidden State (final output):

$$h_t = o_t \odot \tanh(C_t)$$

2.4 How LSTM Solves Vanishing Gradient

Key Mechanism: The cell state C_t acts as a highway

1. **Additive Updates:** $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
 - Addition prevents gradient vanishing (unlike multiplication)
2. **Forget Gate Control:**
 - When $f_t \approx 1$, gradient flows unchanged through time
 - When $f_t \approx 0$, irrelevant information is removed
3. **Gradient Flow:**
 - Gradients can flow backward through cell state without vanishing
 - Only controlled by forget gates (learned, not fixed)

2.5 LSTM Forward Propagation - Detailed Steps

Given:

- Input sequence: $[x_1, x_2, \dots, x_T]$
- Initial states: $h_0 = 0, C_0 = 0$

For $t = 1$ to T :

Step 1: Calculate Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Step 2: Calculate Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Step 3: Calculate Candidate Cell State

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Step 4: Update Cell State

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Step 5: Calculate Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Step 6: Calculate Hidden State

$$h_t = o_t \odot \tanh(C_t)$$

Step 7: Generate Output (if needed)

$$\hat{y}_t = \text{softmax}(W_y \cdot h_t + b_y)$$

2.6 Parameter Count in LSTM

For input dimension d and hidden dimension h :

$$\text{Total parameters} = 4 \times [(d \times h) + (h \times h) + h]$$

Breakdown:

- Each gate (f, i, o) + candidate (C): 4 components
- Each component has:
 - W for input: $d \times h$ parameters
 - U for hidden: $h \times h$ parameters
 - b for bias: h parameters

Example: If $d=3, h=4$:

- Per gate: $(3 \times 4) + (4 \times 4) + 4 = 32$ parameters
- Total: $4 \times 32 = 128$ parameters

2.7 LSTM vs Standard RNN

Feature	RNN	LSTM
Memory	Short-term (hidden state)	Long-term (cell state) + Short-term (hidden state)
Gates	None	3 (Forget, Input, Output)
Parameters	Fewer	More (4× of RNN)
Gradient Flow	Vanishes over time	Maintained through cell state
Long Dependencies	Cannot learn	Can learn
Training	Faster	Slower
Complexity	Simple	Complex

3. GRU (Gated Recurrent Unit)

3.1 Why GRU?

Motivation: LSTM is powerful but complex (many parameters, slow training). GRU simplifies LSTM while maintaining similar performance.

Key Idea: Combine cell state and hidden state into a single hidden state, reduce gates from 3 to 2.

3.2 GRU Architecture

Components:

1. **Hidden State (h_t):** Single state (combines C_t and h_t from LSTM)
2. **Two Gates:**
 - **Reset Gate (r_t):** Decides how much past information to forget
 - **Update Gate (z_t):** Decides how much past information to keep

Note: GRU does NOT have a separate cell state or output gate

3.3 GRU Mathematical Formulation

At each time step t :

1. **Update Gate** (decides balance between old and new):

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

- $z_t \approx 1$: Keep old information
- $z_t \approx 0$: Use new information

2. Reset Gate (decides how much past to forget):

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

- $r_t \approx 0$: Ignore past hidden state
- $r_t \approx 1$: Use past hidden state

3. Candidate Hidden State (new memory content):

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$

- Reset gate modulates how much of h_{t-1} to use

4. Final Hidden State (blend old and new):

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

- First part: contribution from new candidate
- Second part: contribution from old hidden state
- Update gate balances between them

3.4 GRU Forward Propagation - Detailed Steps

Given:

- Input: x_t
- Previous hidden state: h_{t-1}

Steps:

Step 1: Calculate Reset Gate

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

Step 2: Calculate Update Gate

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

Step 3: Calculate Candidate Hidden State

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$

Step 4: Calculate Final Hidden State

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

3.5 How GRU Works - Intuitive Understanding

Scenario: Reading a story

Imagine you're reading: "The cat sat on the mat. It was sleeping."

At "It":

- **Reset Gate (r_t):** "Do I need to remember details about position?"
 - $r_t \approx 0$: No, just focus on subject (cat)
- **Update Gate (z_t):** "Should I update my understanding?"
 - $z_t \approx 0$: Yes, "It" refers to the cat, update memory
- **Candidate $\tilde{h}_t$:** New information = "sleeping cat"
- **Final h_t :** Blend old memory (cat on mat) + new info (sleeping)

3.6 Parameter Count in GRU

For input dimension d and hidden dimension h :

$$\text{Total parameters} = 3 \times [(d \times h) + (h \times h) + h]$$

Breakdown:

- 2 gates (r, z) + 1 candidate ($\tilde{h}$): 3 components
- Each component has same structure as LSTM gate

Example: If $d=3, h=4$:

- Per component: $(3 \times 4) + (4 \times 4) + 4 = 32$ parameters
- Total: $3 \times 32 = 96$ parameters

Comparison:

- LSTM: 128 parameters
- GRU: 96 parameters
- GRU has 25% fewer parameters!

3.7 LSTM vs GRU Comparison

Feature	LSTM	GRU
Number of Gates	3 (Forget, Input, Output)	2 (Reset, Update)
Memory Units	Cell state (C_t) + Hidden state (h_t)	Single hidden state (h_t)
Parameter Count	$4 \times [(d \times h) + (h \times h) + h]$	$3 \times [(d \times h) + (h \times h) + h]$
Computational Cost	Higher	Lower (25% less)
Training Speed	Slower	Faster
Performance	Slightly better on complex tasks	Comparable, better on simpler tasks
Memory	Explicit long-term (C_t) and short-term (h_t)	Combined in single state
Best For	Very long sequences, complex patterns	Shorter sequences, faster training needed

3.8 When to Use GRU vs LSTM?

Use LSTM when:

- You have very long sequences (>100 timesteps)
- Task requires complex long-term memory
- You have lots of training data
- Computational resources are available

Use GRU when:

- Sequences are moderately long (<100 timesteps)
- Training time/resources are limited
- Simpler model is preferred

- Dataset is smaller

In Practice:

- Try GRU first (faster to experiment)
 - If performance insufficient, try LSTM
 - Often GRU gives similar results with less cost
-

4. Step-by-Step Examples with Dataset

Example 1: Sentiment Analysis with RNN

Dataset: Movie reviews (simplified)

Review	Sentiment
"movie was good"	1 (Positive)
"movie was bad"	0 (Negative)
"movie was not good"	0 (Negative)

Vocabulary: {movie: 1, was: 2, good: 3, bad: 4, not: 5}

Step-by-Step Forward Propagation

Input: "movie was good" → [1, 2, 3]

Architecture:

- Input dimension: 5 (vocab size)
- Hidden dimension: 3
- Output dimension: 2 (positive/negative)

Weights (initialized randomly):

```
W_h = [[0.1, 0.2, -0.1],      # 5×3 matrix
        [0.3, -0.1, 0.2],
        [-0.2, 0.1, 0.3],
        [0.2, 0.3, -0.2],
        [-0.1, 0.2, 0.1]]

U_h = [[0.4, -0.2, 0.1],      # 3×3 matrix
        [0.1, 0.3, -0.1],
        [-0.2, 0.1, 0.4]]

W_o = [[0.5, -0.3, 0.2],      # 2×3 matrix
        [-0.2, 0.4, 0.1]]

b_h = [0.1, 0.0, -0.1]       # 3×1 vector
b_o = [0.0, 0.1]             # 2×1 vector
```

Time Step t=1: Input = "movie" (index 1)

1. **One-hot encode:** $x_1 = [1, 0, 0, 0, 0]$
2. **Initialize:** $h_0 = [0, 0, 0]$
3. **Compute hidden state:**

```
h_1 = tanh(W_h · x_1 + U_h · h_0 + b_h)
h_1 = tanh([0.1, 0.3, -0.2] + [0, 0, 0] + [0.1, 0.0, -0.1])
h_1 = tanh([0.2, 0.3, -0.3])
h_1 = [0.197, 0.291, -0.291]
```

Time Step t=2: Input = "was" (index 2)

1. **One-hot encode:** $x_2 = [0, 1, 0, 0, 0]$
2. **Compute hidden state:**

$$h_2 = \tanh(W_h \cdot x_2 + U_h \cdot h_1 + b_h)$$

$$W_h \cdot x_2 = [0.2, -0.1, 0.1] \text{ (2nd row of } W_h)$$

$$\begin{aligned} U_h \cdot h_1 &= \begin{bmatrix} 0.4 & -0.2 & 0.1 \\ 0.1 & 0.3 & -0.1 \\ -0.2 & 0.1 & 0.4 \end{bmatrix} \times \begin{bmatrix} 0.197 \\ 0.291 \\ -0.291 \end{bmatrix} \\ &= [0.0497, 0.1167, -0.0777] \end{aligned}$$

$$h_2 = \tanh([0.2, -0.1, 0.1] + [0.0497, 0.1167, -0.0777] + [0.1, 0.0, -0.1])$$

$$h_2 = \tanh([0.3497, 0.0167, -0.0777])$$

$$h_2 = [0.336, 0.0167, -0.0775]$$

Time Step t=3: Input = "good" (index 3)

1. **One-hot encode:** $x_3 = [0, 0, 1, 0, 0]$

2. **Compute hidden state:**

$$h_3 = \tanh(W_h \cdot x_3 + U_h \cdot h_2 + b_h)$$

$$h_3 = [0.398, 0.124, 0.089]$$

3. **Compute output:**

$$o_3 = W_o \cdot h_3 + b_o$$

$$\begin{aligned} o_3 &= \begin{bmatrix} 0.5 & -0.3 & 0.2 \\ -0.2 & 0.4 & 0.1 \end{bmatrix} \times \begin{bmatrix} 0.398 \\ 0.124 \\ 0.089 \end{bmatrix} + \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix} \\ &= [0.180, 0.159] \end{aligned}$$

$$\hat{y}_3 = \text{softmax}(o_3) = [0.505, 0.495]$$

Interpretation:

- Probability of positive: 0.505
 - Probability of negative: 0.495
 - Prediction: Positive (class 1)
-

Example 2: LSTM Forward Pass (Detailed)

Scenario: Process sequence $[x_1, x_2]$ with LSTM

Given:

- Input dimension: 2
- Hidden dimension: 3
- Initial states: $h_0 = [0, 0, 0]$, $C_0 = [0, 0, 0]$

Input:

```
x_1 = [1, 0]
x_2 = [0, 1]
```

Weights (simplified, 3×5 because $[h, x]$ is concatenated):

```
W_f = [[-0.1, 0.2, 0.3, -0.2, 0.1],
        [0.2, -0.1, 0.1, 0.3, -0.2],
        [0.1, 0.3, -0.2, 0.1, 0.2]]

W_i = [[0.3, -0.2, 0.1, 0.2, -0.1],
        [-0.1, 0.2, 0.3, -0.2, 0.1],
        [0.2, 0.1, -0.1, 0.3, 0.2]]

W_c = [[0.2, 0.1, -0.2, 0.3, -0.1],
        [0.1, -0.2, 0.3, 0.1, 0.2],
        [-0.1, 0.3, 0.1, -0.2, 0.1]]

W_o = [[-0.2, 0.3, 0.1, -0.1, 0.2],
        [0.1, -0.1, 0.2, 0.3, -0.2],
        [0.3, 0.2, -0.2, 0.1, 0.1]]

b_f = [0.0, 0.1, -0.1]
b_i = [0.1, 0.0, 0.1]
b_c = [0.0, 0.0, 0.0]
b_o = [-0.1, 0.1, 0.0]
```

Time Step $t=1$

Concatenate: $[h_0, x_1] = [0, 0, 0, 1, 0]$

Step 1: Forget Gate

$$f_1 = \sigma(W_f \cdot [h_0, x_1] + b_f)$$

$$W_f \cdot [0, 0, 0, 1, 0] = [-0.2, 0.3, 0.1] \quad (4\text{th column})$$

$$f_1 = \sigma([-0.2, 0.3, 0.1] + [0.0, 0.1, -0.1])$$

$$f_1 = \sigma([-0.2, 0.4, 0.0])$$

$$f_1 = [0.450, 0.599, 0.500]$$

Step 2: Input Gate

$$i_1 = \sigma(W_i \cdot [h_0, x_1] + b_i)$$

$$i_1 = \sigma([0.2, -0.2, 0.3] + [0.1, 0.0, 0.1])$$

$$i_1 = \sigma([0.3, -0.2, 0.4])$$

$$i_1 = [0.574, 0.450, 0.599]$$

Step 3: Candidate Cell State

$$\tilde{C}_1 = \tanh(W_c \cdot [h_0, x_1] + b_c)$$

$$\tilde{C}_1 = \tanh([0.3, 0.1, -0.2] + [0.0, 0.0, 0.0])$$

$$\tilde{C}_1 = \tanh([0.3, 0.1, -0.2])$$

$$\tilde{C}_1 = [0.291, 0.100, -0.197]$$

Step 4: Update Cell State

$$C_1 = f_1 \odot C_0 + i_1 \odot \tilde{C}_1$$

$$f_1 \odot C_0 = [0.450, 0.599, 0.500] \odot [0, 0, 0] = [0, 0, 0]$$

$$\begin{aligned} i_1 \odot \tilde{C}_1 &= [0.574, 0.450, 0.599] \odot [0.291, 0.100, -0.197] \\ &= [0.167, 0.045, -0.118] \end{aligned}$$

$$C_1 = [0, 0, 0] + [0.167, 0.045, -0.118]$$

$$C_1 = [0.167, 0.045, -0.118]$$

Step 5: Output Gate

$$o_1 = \sigma(W_o \cdot [h_0, x_1] + b_o)$$

$$o_1 = \sigma([-0.1, 0.3, 0.1] + [-0.1, 0.1, 0.0])$$

$$o_1 = \sigma([-0.2, 0.4, 0.1])$$

$$o_1 = [0.450, 0.599, 0.525]$$

Step 6: Hidden State

```
h_1 = o_1 ⊙ tanh(C_1)
h_1 = [0.450, 0.599, 0.525] ⊙ tanh([0.167, 0.045, -0.118])
h_1 = [0.450, 0.599, 0.525] ⊙ [0.165, 0.045, -0.117]
h_1 = [0.074, 0.027, -0.061]
```

Summary of t=1:

- Cell state: $C_1 = [0.167, 0.045, -0.118]$
- Hidden state: $h_1 = [0.074, 0.027, -0.061]$

Time Step t=2

Concatenate: $[h_1, x_2] = [0.074, 0.027, -0.061, 0, 1]$

Following same steps:

```
f_2 = [0.523, 0.475, 0.549]
i_2 = [0.450, 0.574, 0.549]
C_2 = [0.245, -0.145, 0.197]
C_2 = f_2 ⊙ C_1 + i_2 ⊙ C_2
      = [0.197, 0.038, -0.043]
o_2 = [0.549, 0.450, 0.525]
h_2 = o_2 ⊙ tanh(C_2)
      = [0.106, 0.017, -0.022]
```

Final Output: h_2 can be used for classification/prediction

Example 3: GRU Forward Pass (Detailed)

Same setup as LSTM example

Input:

```
x_1 = [1, 0]
h_0 = [0, 0, 0]
```

Weights (3×5 matrices):

```
W_z = [[0.2, -0.1, 0.3, 0.1, -0.2],
        [0.1, 0.2, -0.1, 0.3, 0.1],
        [-0.1, 0.3, 0.2, -0.2, 0.1]]
```

```
W_r = [[-0.2, 0.3, 0.1, -0.1, 0.2],
        [0.3, -0.1, 0.2, 0.1, -0.2],
        [0.1, 0.2, -0.2, 0.3, 0.1]]
```

```
W_h = [[0.3, -0.2, 0.1, 0.2, -0.1],
        [-0.1, 0.2, 0.3, -0.2, 0.1],
        [0.2, 0.1, -0.1, 0.3, 0.2]]
```

```
b_z = [0.0, 0.1, -0.1]
```

```
b_r = [0.1, 0.0, 0.1]
```

```
b_h = [0.0, 0.0, 0.0]
```

Time Step t=1

Concatenate: $[h_0, x_1] = [0, 0, 0, 1, 0]$

Step 1: Update Gate

```
z_1 =  $\sigma(W_z \cdot [h_0, x_1] + b_z)$ 
z_1 =  $\sigma([0.1, 0.3, -0.2] + [0.0, 0.1, -0.1])$ 
z_1 =  $\sigma([0.1, 0.4, -0.3])$ 
z_1 = [0.525, 0.599, 0.426]
```

Step 2: Reset Gate

```
r_1 =  $\sigma(W_r \cdot [h_0, x_1] + b_r)$ 
r_1 =  $\sigma([-0.1, 0.1, 0.3] + [0.1, 0.0, 0.1])$ 
r_1 =  $\sigma([0.0, 0.1, 0.4])$ 
r_1 = [0.500, 0.525, 0.599]
```

Step 3: Modulated Hidden State

```
r_1  $\odot$  h_0 = [0.500, 0.525, 0.599]  $\odot$  [0, 0, 0] = [0, 0, 0]
```

Step 4: Candidate Hidden State

Concatenate: $[r_1 \odot h_0, x_1] = [0, 0, 0, 1, 0]$

$\tilde{h}_1 = \tanh(W_h \cdot [r_1 \odot h_0, x_1] + b_h)$

$\tilde{h}_1 = \tanh([0.2, -0.2, 0.3] + [0, 0, 0])$

$\tilde{h}_1 = \tanh([0.2, -0.2, 0.3])$

$\tilde{h}_1 = [0.197, -0.197, 0.291]$

Step 5: Final Hidden State

$h_1 = (1 - z_1) \odot \tilde{h}_1 + z_1 \odot h_0$

$(1 - z_1) = [0.475, 0.401, 0.574]$

$(1 - z_1) \odot \tilde{h}_1 = [0.475, 0.401, 0.574] \odot [0.197, -0.197, 0.291]$
 $= [0.094, -0.079, 0.167]$

$z_1 \odot h_0 = [0.525, 0.599, 0.426] \odot [0, 0, 0] = [0, 0, 0]$

$h_1 = [0.094, -0.079, 0.167] + [0, 0, 0]$

$h_1 = [0.094, -0.079, 0.167]$

Interpretation:

- Update gate z_1 determines balance between old and new
- Since $h_0 = 0$, h_1 comes entirely from candidate $\tilde{h}_1$
- Reset gate had no effect (h_0 was zero)

Time Step $t=2$

$[h_1, x_2] = [0.094, -0.079, 0.167, 0, 1]$

$z_2 = [0.549, 0.525, 0.450]$

$r_2 = [0.574, 0.525, 0.549]$

$r_2 \odot h_1 = [0.054, -0.041, 0.092]$

$\tilde{h}_2 = \tanh(W_h \cdot [r_2 \odot h_1, x_2] + b_h)$

$\tilde{h}_2 = [0.167, 0.291, -0.145]$

$h_2 = (1 - z_2) \odot \tilde{h}_2 + z_2 \odot h_1$

$h_2 = [0.127, 0.096, -0.005]$

5. Answers to Exam Questions

Question 1: Why does CNN process images but not sequential data?

Answer:

CNNs are designed for **spatial data** (images) where:

1. **Local patterns matter:** Features like edges, textures are detected through kernels/filters
2. **Translation invariance:** Same feature can appear anywhere in image
3. **Fixed input size:** Images have fixed dimensions (height $\times$ width $\times$ channels)
4. **No temporal dependency:** Pixels don't have sequential relationships

CNNs **cannot** process sequential data because:

1. **Order doesn't matter in convolution:** Kernel slides spatially, not temporally
2. **No memory mechanism:** Each convolution is independent
3. **Fixed-size windows:** Cannot handle variable-length sequences

For sequential data, we need:

- **Temporal dependencies:** "I ate" $\rightarrow$ "an apple" (order matters)
- **Memory:** Remember context from earlier in sequence
- **Variable length:** Sentences/time-series have different lengths
- **RNN provides this** through recurrent connections

Question 2: What is the main part that solves the weight update problem in ML?

Answer:

The **Backpropagation** algorithm with **Gradient Descent** optimization.

Core Mechanism:

1. **Core Block:** Weighted Average
 - Each perceptron computes: $y = \sum(w_i \times x_i) + b$
2. **Problem:** How to find optimal weights?
3. **Solution:**
 - **Backpropagation:** Computes gradients ($\partial \text{Loss} / \partial w$) by chain rule

- **Gradient Descent:** Updates weights in direction that reduces loss

$$w_{\text{new}} = w_{\text{old}} - \text{learning_rate} \times \partial \text{Loss} / \partial w$$

For single perceptron:

- Cannot capture hierarchical features
- Solution: Stack multiple layers

Why hierarchical features matter:

- Layer 1: Low-level features (edges)
 - Layer 2: Mid-level features (shapes)
 - Layer 3: High-level features (objects)
 - This creates 2-bit, 4-bit, 8-bit feature complexity hierarchy
-

Question 3: Patchwise Image Processing

Answer:

What is patchwise processing? CNNs process images through **consolidated/local space** (patches) rather than whole image at once.

Why patches?

1. **Kernel Use:** Uses sliding window/kernel approach
2. **Local feature extraction:** Detects patterns in small regions
3. **Convolution:** Calculates overlapping area (element-wise multiplication + sum)

$$\text{Output} = \sum (F(x) * g(x))$$

where $F(x)$ is filter/kernel, $g(x)$ is image patch

Advantages:

1. **Parameter sharing:** Same kernel used across image
 2. **Translation invariance:** Detects features anywhere
 3. **Computational efficiency:** Process patches, not full image
-

6. Mathematical Problems for Practice

Problem 1: RNN Forward Propagation (Complete Calculation)

Given:

- Vocabulary: {I:1, love:2, ML:3}
- Input sequence: "I love ML"
- Architecture:
 - Vocab size: 3
 - Hidden units: 2
 - Output units: 1 (sentiment: 0=negative, 1=positive)

Weights:

```
W_h = [[0.5, -0.3],      # 3×2
        [0.2, 0.4],
        [-0.1, 0.3]]
```

```
U_h = [[0.6, -0.2],      # 2×2
        [0.3, 0.5]]
```

```
W_o = [[0.8, -0.4]]      # 1×2
```

```
b_h = [0.1, 0.0]
```

```
b_o = [0.2]
```

Tasks: a) Perform forward propagation for all 3 time steps b) Calculate the final output after processing "I love ML" c) If true label is 1 (positive), calculate the loss d) Calculate $\partial L / \partial W_o$ (gradient for output weights)

Solution:

Time t=1: Input "I" $\rightarrow x_1 = [1, 0, 0]$

Initialize: $h_0 = [0, 0]$

$$h_1 = \tanh(W_h \cdot x_1 + U_h \cdot h_0 + b_h)$$

$$W_h \cdot x_1 = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \\ -0.1 & 0.3 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.2 \\ -0.1 \end{bmatrix}$$

$$\text{Taking first row: } [0.5, -0.3, -0.1] \cdot [1, 0, 0] = 0.5$$

Actually, W_h is 3×2 , x_1 is 3×1 , so:

$$W_h^T \cdot x_1 = \begin{bmatrix} 0.5 & 0.2 & -0.1 \\ -0.3 & 0.4 & 0.3 \end{bmatrix} \times \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.5 \\ -0.3 \end{bmatrix}$$

$$h_1 = \tanh([0.5, -0.3] + [0, 0] + [0.1, 0.0])$$

$$h_1 = \tanh([0.6, -0.3])$$

$$h_1 = [0.537, -0.291]$$

Time t=2: Input "love" $\rightarrow x_2 = [0, 1, 0]$

$$W_h^T \cdot x_2 = \begin{bmatrix} 0.5 & 0.2 & -0.1 \\ -0.3 & 0.4 & 0.3 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$

$$U_h \cdot h_1 = \begin{bmatrix} 0.6 & -0.2 \\ 0.3 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0.537 \\ -0.291 \end{bmatrix} = \begin{bmatrix} 0.380 \\ 0.016 \end{bmatrix}$$

$$h_2 = \tanh([0.2, 0.4] + [0.380, 0.016] + [0.1, 0.0])$$

$$h_2 = \tanh([0.680, 0.416])$$

$$h_2 = [0.592, 0.394]$$

Time t=3: Input "ML" $\rightarrow x_3 = [0, 0, 1]$

$$W_h^T \cdot x_3 = \begin{bmatrix} 0.5 & 0.2 & -0.1 \\ -0.3 & 0.4 & 0.3 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.1 \\ 0.3 \end{bmatrix}$$

$$U_h \cdot h_2 = \begin{bmatrix} 0.6 & -0.2 \\ 0.3 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0.592 \\ 0.394 \end{bmatrix} = \begin{bmatrix} 0.276 \\ 0.375 \end{bmatrix}$$

$$h_3 = \tanh([-0.1, 0.3] + [0.276, 0.375] + [0.1, 0.0])$$

$$h_3 = \tanh([0.276, 0.675])$$

$$h_3 = [0.270, 0.589]$$

Output Calculation:

$$\begin{aligned}o_3 &= W_o \cdot h_3 + b_o \\o_3 &= [0.8, -0.4] \cdot [0.270, 0.589] + 0.2 \\o_3 &= (0.8 \times 0.270) + (-0.4 \times 0.589) + 0.2 \\o_3 &= 0.216 - 0.236 + 0.2 \\o_3 &= 0.180 \\\hat{y} &= \sigma(o_3) = \sigma(0.180) = 0.545\end{aligned}$$

Loss Calculation (Binary Cross-Entropy):

$$\begin{aligned}L &= -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})] \\L &= -[1 \cdot \log(0.545) + 0 \cdot \log(0.455)] \\L &= -\log(0.545) \\L &= 0.607\end{aligned}$$

Gradient Calculation:

$$\begin{aligned}\partial L / \partial o_3 &= \hat{y} - y = 0.545 - 1 = -0.455 \\\partial L / \partial W_o &= (\partial L / \partial o_3) \cdot h_3^T \\\partial L / \partial W_o &= -0.455 \cdot [0.270, 0.589] \\\partial L / \partial W_o &= [-0.123, -0.268]\end{aligned}$$

Weight Update (with learning rate $\eta = 0.1$):

$$\begin{aligned}W_{o_new} &= W_o - \eta \cdot \partial L / \partial W_o \\W_{o_new} &= [0.8, -0.4] - 0.1 \cdot [-0.123, -0.268] \\W_{o_new} &= [0.8, -0.4] + [0.0123, 0.0268] \\W_{o_new} &= [0.812, -0.373]\end{aligned}$$

Problem 2: LSTM Gate Calculations

Given:

- Input: $x_t = [0.5, 0.8]$
- Previous states: $h_{\{t-1\}} = [0.3, -0.2, 0.4]$, $C_{\{t-1\}} = [0.6, -0.3, 0.5]$
- Dimensions: input=2, hidden=3

Weights (simplified, 3×5 for $[h, x]$ concatenation):

```
W_f = [[0.2, -0.1, 0.3, 0.4, -0.2],  
        [-0.1, 0.2, 0.1, -0.3, 0.2],  
        [0.3, 0.1, -0.2, 0.2, 0.1]]
```

```
W_i = [[-0.2, 0.3, 0.1, -0.1, 0.4],  
        [0.1, -0.2, 0.3, 0.2, -0.1],  
        [0.2, 0.1, -0.1, 0.3, 0.2]]
```

```
W_c = [[0.3, -0.2, 0.1, 0.2, -0.3],  
        [-0.1, 0.2, 0.3, -0.2, 0.1],  
        [0.2, 0.1, -0.2, 0.1, 0.3]]
```

```
W_o = [[-0.1, 0.2, 0.3, -0.2, 0.1],  
        [0.2, -0.1, 0.1, 0.3, -0.2],  
        [0.1, 0.3, -0.2, 0.1, 0.2]]
```

All biases = $[0, 0, 0]$

Tasks: a) Calculate forget gate f_t b) Calculate input gate i_t c) Calculate candidate $\tilde{C}_t$ d) Calculate new cell state C_t e) Calculate output gate o_t f) Calculate new hidden state h_t

Solution:

Concatenate inputs: $[h_{t-1}, x_t] = [0.3, -0.2, 0.4, 0.5, 0.8]$

a) Forget Gate:

```
f_t =  $\sigma(W_f \cdot [h_{t-1}, x_t])$ 
```

```
Row 1:  $[0.2, -0.1, 0.3, 0.4, -0.2] \cdot [0.3, -0.2, 0.4, 0.5, 0.8]$   
      =  $0.06 + 0.02 + 0.12 + 0.20 - 0.16 = 0.24$ 
```

```
Row 2:  $[-0.1, 0.2, 0.1, -0.3, 0.2] \cdot [0.3, -0.2, 0.4, 0.5, 0.8]$   
      =  $-0.03 - 0.04 + 0.04 - 0.15 + 0.16 = -0.02$ 
```

```
Row 3:  $[0.3, 0.1, -0.2, 0.2, 0.1] \cdot [0.3, -0.2, 0.4, 0.5, 0.8]$   
      =  $0.09 - 0.02 - 0.08 + 0.10 + 0.08 = 0.17$ 
```

```
f_t =  $\sigma([0.24, -0.02, 0.17])$ 
```

```
f_t =  $[0.560, 0.495, 0.542]$ 
```

b) Input Gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t])$$

$$\begin{aligned} \text{Row 1: } & [-0.2, 0.3, 0.1, -0.1, 0.4] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = -0.06 - 0.06 + 0.04 - 0.05 + 0.32 = 0.19 \end{aligned}$$

$$\begin{aligned} \text{Row 2: } & [0.1, -0.2, 0.3, 0.2, -0.1] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = 0.03 + 0.04 + 0.12 + 0.10 - 0.08 = 0.21 \end{aligned}$$

$$\begin{aligned} \text{Row 3: } & [0.2, 0.1, -0.1, 0.3, 0.2] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = 0.06 - 0.02 - 0.04 + 0.15 + 0.16 = 0.31 \end{aligned}$$

$$i_t = \sigma([0.19, 0.21, 0.31])$$

$$i_t = [0.547, 0.552, 0.577]$$

c) Candidate Cell State:

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t])$$

$$\begin{aligned} \text{Row 1: } & [0.3, -0.2, 0.1, 0.2, -0.3] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = 0.09 + 0.04 + 0.04 + 0.10 - 0.24 = 0.03 \end{aligned}$$

$$\begin{aligned} \text{Row 2: } & [-0.1, 0.2, 0.3, -0.2, 0.1] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = -0.03 - 0.04 + 0.12 - 0.10 + 0.08 = 0.03 \end{aligned}$$

$$\begin{aligned} \text{Row 3: } & [0.2, 0.1, -0.2, 0.1, 0.3] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = 0.06 - 0.02 - 0.08 + 0.05 + 0.24 = 0.25 \end{aligned}$$

$$\tilde{C}_t = \tanh([0.03, 0.03, 0.25])$$

$$\tilde{C}_t = [0.030, 0.030, 0.245]$$

d) New Cell State:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$\begin{aligned} f_t \odot C_{t-1} &= [0.560, 0.495, 0.542] \odot [0.6, -0.3, 0.5] \\ &= [0.336, -0.149, 0.271] \end{aligned}$$

$$\begin{aligned} i_t \odot \tilde{C}_t &= [0.547, 0.552, 0.577] \odot [0.030, 0.030, 0.245] \\ &= [0.016, 0.017, 0.141] \end{aligned}$$

$$C_t = [0.336, -0.149, 0.271] + [0.016, 0.017, 0.141]$$

$$C_t = [0.352, -0.132, 0.412]$$

e) Output Gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t])$$

$$\begin{aligned}\text{Row 1: } & [-0.1, 0.2, 0.3, -0.2, 0.1] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = -0.03 - 0.04 + 0.12 - 0.10 + 0.08 = 0.03\end{aligned}$$

$$\begin{aligned}\text{Row 2: } & [0.2, -0.1, 0.1, 0.3, -0.2] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = 0.06 + 0.02 + 0.04 + 0.15 - 0.16 = 0.11\end{aligned}$$

$$\begin{aligned}\text{Row 3: } & [0.1, 0.3, -0.2, 0.1, 0.2] \cdot [0.3, -0.2, 0.4, 0.5, 0.8] \\ & = 0.03 - 0.06 - 0.08 + 0.05 + 0.16 = 0.10\end{aligned}$$

$$o_t = \sigma([0.03, 0.11, 0.10])$$

$$o_t = [0.508, 0.527, 0.525]$$

f) New Hidden State:

$$h_t = o_t \odot \tanh(C_t)$$

$$\begin{aligned}\tanh(C_t) &= \tanh([0.352, -0.132, 0.412]) \\ &= [0.338, -0.131, 0.390]\end{aligned}$$

$$h_t = [0.508, 0.527, 0.525] \odot [0.338, -0.131, 0.390]$$

$$h_t = [0.172, -0.069, 0.205]$$

Summary:

$f_t = [0.560, 0.495, 0.542]$ ← Forget 44-50% of old memory
 $i_t = [0.547, 0.552, 0.577]$ ← Add 55-58% of new information
 $C_t = [0.352, -0.132, 0.412]$ ← Updated long-term memory
 $o_t = [0.508, 0.527, 0.525]$ ← Output 51-53% of cell state
 $h_t = [0.172, -0.069, 0.205]$ ← New hidden state (short-term memory)

Conclusion

This study guide covers:

- ✓ Complete theory of RNN, LSTM, and GRU
- ✓ Mathematical formulations with explanations
- ✓ Step-by-step examples with actual calculations

- ✓ Answers to exam questions
- ✓ Practice problems with detailed solutions

Key Takeaways for Exam:

1. **RNN:** Sequential data, memory through hidden state, suffers from vanishing gradients
2. **LSTM:** Solves long-term dependency problem with:
 - Cell state (highway for gradients)
 - 3 gates (forget, input, output)
 - 4 parameter sets (more complex)
3. **GRU:** Simplified LSTM with:
 - Single hidden state (no separate cell state)
 - 2 gates (reset, update)
 - 3 parameter sets (25% fewer parameters)
 - Often similar performance to LSTM
4. **When to use what:**
 - Short sequences, simple tasks → RNN
 - Long sequences, complex patterns → LSTM
 - Need faster training, limited resources → GRU

Exam Tips:

- Understand gate mechanisms (what each gate does)
- Know parameter counts (4 vs 3 for LSTM vs GRU)
- Practice forward propagation by hand
- Understand why LSTM/GRU solve vanishing gradients
- Know applications and architecture types

Good luck with your exam! 🎓